

Playwright Cheat Sheet — TypeScript

Quick reference for interviews and real-world automation frameworks

1. Setup & Test Commands

```
npm init playwright@latest
npx playwright test
npx playwright test --headed
npx playwright test --debug
npx playwright test --ui
npx playwright test login.spec.ts
npx playwright test -g "Login"
npx playwright test --project=chromium
npx playwright show-report
```

2. Basic Test & Hooks

```
import { test, expect } from '@playwright/test';

test('Login test', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});

test.beforeEach(async ({ page }) => {
  await page.goto('/login');
});
test.afterEach(async ({ page }) => {
  // cleanup
});
```

3. Locators

```
page.getByRole('button', { name: 'Login' });
page.getByText('Welcome');
page.getByLabel('Username');
page.getByPlaceholder('Enter username');
page.getByTestId('login-button');

page.locator('#username');
page.locator('.login-button');
page.locator('input[name="username"]');
page.locator('//input[@name="username"]');
```

4. Actions

```
await page.getByRole('button', { name: 'Login' }).click();
await page.getByLabel('Username').fill('admin');
await page.locator('#username').pressSequentially('admin');
await page.locator('#username').clear();
await page.getByLabel('Remember me').check();
await page.getByLabel('Remember me').uncheck();
await page.getByLabel('Male').check();
await page.getByLabel('Country').selectOption({ label: 'India' });
```

5. Assertions

```
await expect(page).toHaveTitle('Login');
await expect(page).toHaveURL(/dashboard/);
await expect(page.getByText('Welcome')).toBeVisible();
await expect(page.getByText('Welcome')).toBeHidden();
await expect(page.getByRole('button')).toBeEnabled();
await expect(page.getByRole('button')).toBeDisabled();
await expect(page.locator('#username')).toHaveValue('admin');
await expect(page.locator('.message')).toContainText('Success');
```

6. Waits

```
// Prefer Playwright auto-waiting and web-first assertions.
await expect(page.getByText('Success')).toBeVisible();
```

```
await page.waitForURL('**/dashboard');
await page.waitForLoadState('networkidle');
await page.locator('#result').waitFor();
```

7. Navigation

```
await page.goto('https://example.com');
await page.goBack();
await page.goForward();
await page.reload();
```

8. Browser / Context / Page ■■■■

```
Browser
  ↓
BrowserContext
  ↓
Page
```

```
const browser = await chromium.launch();
const context = await browser.newContext();
const page = await context.newPage();
await page.goto('https://example.com');
```

9. Multiple Tabs / Windows ■■■■

```
const newPagePromise = context.waitForEvent('page');
await page.getByText('Open Window').click();
const newPage = await newPagePromise;
await newPage.waitForLoadState();

const popupPromise = page.waitForEvent('popup');
await page.getByText('Open').click();
const popup = await popupPromise;
```

10. Frames / iFrames

```
const frame = page.frameLocator('#payment-frame');
await frame.getByLabel('Card Number').fill('123456789');

const nested = page.frameLocator('#outer-frame')
  .frameLocator('#inner-frame');
await nested.getByRole('button').click();
```

11. Alerts / Dialogs

```
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept();
});

// dismiss: await dialog.dismiss();
// prompt: await dialog.accept('My answer');
```

12. Screenshots

```
await page.screenshot({ path: 'screenshots/login.png' });
await page.screenshot({ path: 'fullpage.png', fullPage: true });
await page.locator('#logo').screenshot({ path: 'logo.png' });
```

13. Video & Trace

```
use: {
  video: 'retain-on-failure',
  trace: 'retain-on-failure'
}

npx playwright show-trace trace.zip
```

14. Mouse & Keyboard

```
await page.mouse.click(100, 200);
await page.mouse.dblclick(100, 200);
await page.mouse.move(100, 200);
```

```

await page.keyboard.press('Enter');
await page.keyboard.press('Control+A');
await page.keyboard.press('Escape');

```

15. File Upload & Download

```

await page.locator('input[type="file"]')
  .setInputFiles('test-data/sample.pdf');

const downloadPromise = page.waitForEvent('download');
await page.getByText('Download').click();
const download = await downloadPromise;
await download.saveAs('downloads/file.pdf');

```

16. API Testing ■■■■

```

test('GET API', async ({ request }) => {
  const response = await request.get('/users');
  expect(response.status()).toBe(200);
  const body = await response.json();
});

const response = await request.post('/users', {
  data: { name: 'Anil', role: 'QA' }
});

const authResponse = await request.get('/users', {
  headers: { Authorization: `Bearer ${token}` }
});

```

17. Network Interception ■■■■

```

await page.route('**/api/users', async route => {
  await route.fulfill({
    status: 200,
    body: JSON.stringify({ name: 'Test User' })
  });
});

await page.route('**/*.png', route => route.abort());
await page.route('**/api/**', route => route.continue());

```

18. Page Events

```

page.on('console', msg => console.log(msg.text()));
page.on('request', request => console.log(request.url()));
page.on('response', response => console.log(response.status()));

```

19. Page Object Model (POM) ■■■■

```

import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly username: Locator;
  readonly password: Locator;
  readonly loginButton: Locator;

  constructor(private page: Page) {
    this.username = page.getByLabel('Username');
    this.password = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
  }

  async login(username: string, password: string) {
    await this.username.fill(username);
    await this.password.fill(password);
    await this.loginButton.click();
  }
}

```

20. Fixtures ■■■■

```

import { test as base } from '@playwright/test';

type MyFixtures = { loginPage: LoginPage };

```

```
export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  }
});
```

21. Configuration

```
export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  retries: 2,
  workers: 4,
  reporter: [['html'], ['list']],
  use: {
    baseUrl: 'https://example.com',
    headless: true,
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'retain-on-failure'
  },
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } }
  ]
});
```

22. Parallel Execution

```
test.describe.configure({ mode: 'parallel' });
```

```
# CLI
npx playwright test --workers=4
```

23. Tags

```
test('Login @smoke', async ({ page }) => {});
```

```
npx playwright test --grep @smoke
npx playwright test --grep-invert @regression
```

24. Environment Variables

```
.env
BASE_URL=https://example.com
USERNAME=admin
PASSWORD=secret
```

```
// TypeScript
process.env.BASE_URL
```

25. Authentication / Storage State ■■■■

```
await context.storageState({
  path: 'auth/user.json'
});

use: {
  storageState: 'auth/user.json'
}
```

26. Data-Driven Testing

```
const users = [
  { username: 'user1', password: 'pass1' },
  { username: 'user2', password: 'pass2' }
];

for (const user of users) {
  test(`Login ${user.username}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill(user.username);
    await page.getByLabel('Password').fill(user.password);
  });
}
```

27. TypeScript Essentials

```
let name: string = 'Anil';
let age: number = 35;
let active: boolean = true;

interface User {
  name: string;
  age: number;
}

function login(username: string, password: string): void {}

async function loginAsync(): Promise<void> {
  await page.goto('/login');
}
```

28. CI/CD

```
npm ci
npx playwright install --with-deps
npx playwright test
npx playwright show-report
```

Typical flow:
Developer → Git → Jenkins/GitHub Actions
→ Install → Playwright Tests → Report → Artifacts

29. Debugging

```
npx playwright test --debug
await page.pause();
npx playwright codegen https://example.com
npx playwright show-trace trace.zip
```

30. Senior-Level Interview Focus ■■■

Playwright architecture
TypeScript fundamentals
Locators and auto-waiting
POM and Fixtures
Authentication / storageState
API testing
Network mocking/interception
Parallel execution and retries
CI/CD
Trace Viewer and debugging
Database validation
Framework design and maintainability
Docker / cloud execution
Test strategy and quality engineering

Quick Comparison

Topic	Playwright
Wait	Auto-waiting + web-first assertions
Locator	getByRole, getByLabel, getByText, getByTestId
Isolation	BrowserContext
Tabs	Multiple Page objects
iframe	frameLocator()
API	request fixture
Mocking	page.route()
Parallel	Workers
Debugging	Trace Viewer

Topic	Playwright
Authentication	storageState
Reporting	HTML / JUnit / other reporters
Language	TypeScript / JavaScript / Java / Python / .NET

Note: Examples are concise reference snippets; adapt imports, URLs, fixtures, and environment configuration to your project.